

CS202: Software Tools and Techniques for CSE

Lecture 6

Shouvick Mondal

shouvick.mondal@iitgn.ac.in

January 2025

Indian Patent 386511 (lecture scope: a subset ONLY)



सत्यमेव जयते

भारत सरकार
GOVERNMENT OF INDIA

पेटेंट कार्यालय
THE PATENT OFFICE

पेटेंट प्रमाणपत्र
PATENT CERTIFICATE
(Rule 74 Of The Patents Rules)

क्रमांक : 044137443
SL No :



पेटेंट सं. / Patent No.	:	386511
आवेदन सं. / Application No.	:	202141018121
फाइल करने की तारीख / Date of Filing	:	19/04/2021
पेटेंटी / Patentee	:	INDIAN INSTITUTE OF TECHNOLOGY MADRAS (IIT MADRAS)
आविष्कारक (जहाँ लागू हो) / Inventor(s)	:	1.SHOUVICK MONDAL 2.RUPESH NASRE

प्रमाणित किया जाता है कि पेटेंटी को उपरोक्त आवेदन में यथाप्रकटित A method for consensus prioritization of regression test-cases supported by parallel execution windows नामक आविष्कार के लिए, पेटेंट अधिनियम, १९७० के उपबंधों के अनुसार आज तारीख 19th day of April 2021 से बीस वर्ष की अवधि के लिए पेटेंट अनुदत्त किया गया है।

The setting: software evolution

```
1 // v0.c
2 #include<stdio.h>
3 ...
4 int main()
5 {
6     int n;
7     scanf("%d", &n);
8     if (n >= 0)
9     {
10         ...
11     }
12     else
13         ...
14 }
15 ...

1 // v1.c
2 #include<stdio.h>
3 ...
4 int main()
5 {
6     int n;
7     scanf("%d", &n);
8     if (n >= 0)
9     {
10         ...
11         n = n - 32000; //change
12     }
13     else
14         ...
15 }
16 ...
```

Figure: Original source code (**v0.c**) and its new version (**v1.c**).

The setting: software evolution

```
1 // v0.c
2 #include<stdio.h>
3 ...
4 int main()
5 {
6     int n;
7     scanf("%d", &n);
8     if (n >= 0)
9     {
10         ...
11     }
12     else
13         ...
14 }
15 ...

1 // v1.c
2 #include<stdio.h>
3 ...
4 int main()
5 {
6     int n;
7     scanf("%d", &n);
8     if (n >= 0)
9     {
10         ...
11         n = n - 32000; //change
12     }
13     else
14         ...
15 }
16 ...
```

Figure: Original source code (v0.c) and its new version (v1.c).

What are we testing for?

Failure: An event that occurs when current service deviates from expected service.

The setting: software evolution

```
1 // v0.c
2 #include<stdio.h>
3 ...
4 int main()
5 {
6     int n;
7     scanf("%d", &n);
8     if (n >= 0)
9     {
10         ...
11     }
12     else
13         ...
14 }
15 ...

1 // v1.c
2 #include<stdio.h>
3 ...
4 int main()
5 {
6     int n;
7     scanf("%d", &n);
8     if (n >= 0)
9     {
10         ...
11         n = n - 32000; //change
12     }
13     else
14         ...
15 }
16 ...
```

Figure: Original source code (v0.c) and its new version (v1.c).

What are we testing for?

Failure: An event that occurs when current service deviates from expected service.

Error: A system state or configuration that might cause a failure.

The setting: software evolution

```
1 // v0.c
2 #include<stdio.h>
3 ...
4 int main()
5 {
6     int n;
7     scanf("%d", &n);
8     if (n >= 0)
9     {
10         ...
11     }
12     else
13         ...
14 }
15 ...

1 // v1.c
2 #include<stdio.h>
3 ...
4 int main()
5 {
6     int n;
7     scanf("%d", &n);
8     if (n >= 0)
9     {
10         ...
11         n = n - 32000; //change
12     }
13     else
14         ...
15 }
16 ...
```

Figure: Original source code (v0.c) and its new version (v1.c).

What are we testing for?

Failure: An event that occurs when current service deviates from expected service.

Error: A system state or configuration that might cause a failure.

Fault: The root cause which gives rise to an error.

The setting: software evolution

```
1 // v0.c
2 #include<stdio.h>
3 ...
4 int main()
5 {
6     int n;
7     scanf("%d", &n);
8     if (n >= 0)
9     {
10         ...
11     }
12     else
13         ...
14 }
15 ...

1 // v1.c
2 #include<stdio.h>
3 ...
4 int main()
5 {
6     int n;
7     scanf("%d", &n);
8     if (n >= 0)
9     {
10         ...
11         n = n - 32000; //change
12     }
13     else
14         ...
15 }
16 ...
```

Figure: Original source code (*v0.c*) and its new version (*v1.c*).

What are we testing for?

Failure: An event that occurs when current service deviates from expected service.

Error: A system state or configuration that might cause a failure.

Fault: The root cause which gives rise to an error.

Testing may consume over 50% of development time.¹

¹

Source: A. Bertolino, *Software Testing Research: Achievements, Challenges, Dreams*, FOSE 2007.

Motivation: parallelization of regression testing

- Three phases of regression testing
 - Offline: Code instrumentation and test-coverage map generation {test-cases $\rightarrow$ code-elements}.

Motivation: parallelization of regression testing

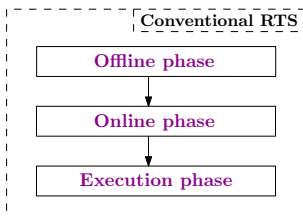
- Three phases of regression testing
 - **Offline**: Code instrumentation and test-coverage map generation {test-cases $\rightarrow$ code-elements}.
 - **Online**: Identify changes, and select regression tests.

Motivation: parallelization of regression testing

- Three phases of regression testing
 - **Offline**: Code instrumentation and test-coverage map generation {test-cases $\rightarrow$ code-elements}.
 - **Online**: Identify changes, and select regression tests.
 - **Execution**: Execute selected tests.

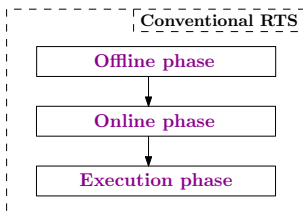
Motivation: parallelization of regression testing

- Three phases of regression testing
 - **Offline**: Code instrumentation and test-coverage map generation {test-cases $\rightarrow$ code-elements}.
 - **Online**: Identify changes, and select regression tests.
 - **Execution**: Execute selected tests.



Motivation: parallelization of regression testing

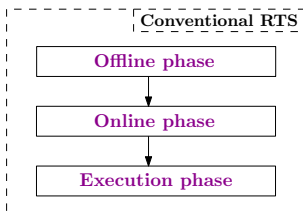
- Three phases of regression testing
 - **Offline**: Code instrumentation and test-coverage map generation {test-cases → code-elements}.
 - **Online**: Identify changes, and select regression tests.
 - **Execution**: Execute selected tests.



None of the existing works target phase-wise parallelization!

Motivation: parallelization of regression testing

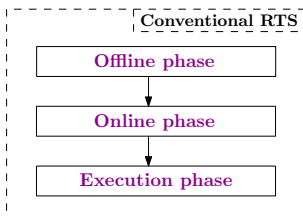
- Three phases of regression testing
 - **Offline**: Code instrumentation and test-coverage map generation {test-cases $\rightarrow$ code-elements}.
 - **Online**: Identify changes, and select regression tests.
 - **Execution**: Execute selected tests.



None of the existing works target phase-wise parallelization!
We propose *Mahtab* to achieve this using parallelization windows.

Motivation: parallelization of regression testing

- Three phases of regression testing
 - **Offline**: Code instrumentation and test-coverage map generation {test-cases $\rightarrow$ code-elements}.
 - **Online**: Identify changes, and select regression tests.
 - **Execution**: Execute selected tests.



None of the existing works target phase-wise parallelization!

We propose *Mahtab* to achieve this using parallelization windows.

Assumption: All test-cases are *independent*² and can be executed in *parallel*.

²

Source: M. J. Harrold et al., *Regression Test Selection for Java Software*, OOPSLA 2001.

Regression testing with parallelization windows

Sequential versus parallel execution windows

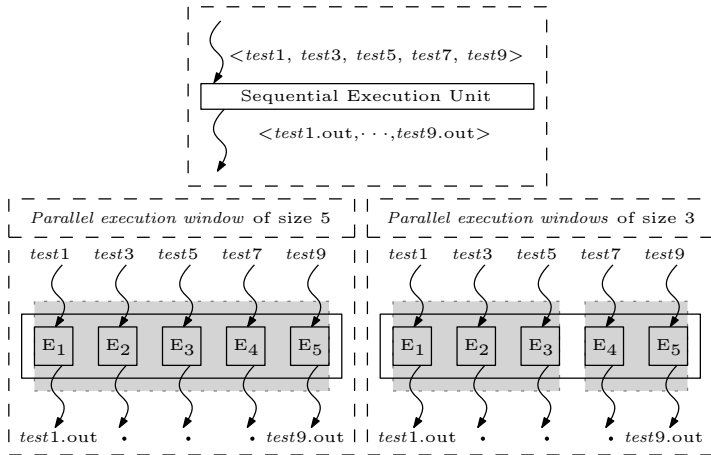


Figure: Executing selected tests: sequential versus parallel windows (shaded in gray).

Our approach: Mahtab

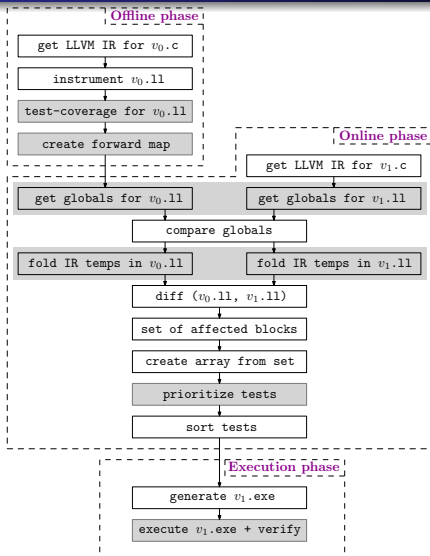
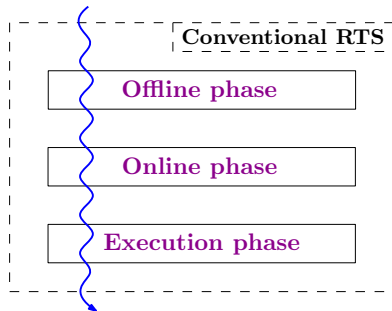
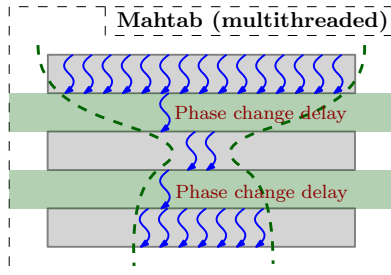


Figure: Module dependency graph of Mahtab for offline, online, and execution phases. The shaded blocks are **individually** or in **group** executed in **parallel**.

Parallelism style of Mahtab

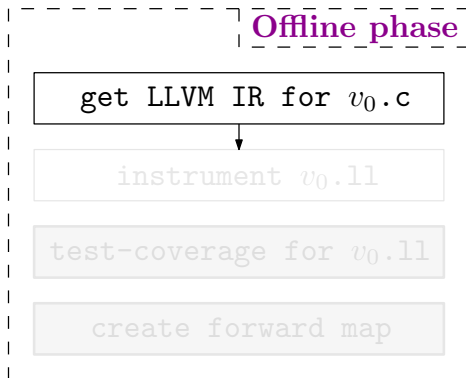


Sequential



Ice-cream-cup parallelism

Parallelizing offline phase



Source to intermediate code generation

```
1 // v0.c
2 #include<stdio.h>
3 ...
4 int main()
5 {
6     int n;
7     scanf("%d", &n);
8     if (n >= 0)
9     {
10         ...
11     }
12     else
13     {
14         ...
15     }
16 }
```

```
1 ;v0.ll
2 ...
3 global_segment
4 ...
5 define i32 @main() #0 {
6 entry:
7     ...
8 if.then: ; preds = %entry
9     ...
10 if.else: ; preds = %entry
11     ...
12 if.end: ; preds = %if.else, %if.then
13     ...
14 }
15 ...
```

Figure: LLVM IR generation: *v0.c* $\rightarrow$ *v0.ll*.

Offline phase

get LLVM IR for $v_0.c$



instrument $v_0.ll$



test-coverage for $v_0.ll$

create forward map

Instrumenting the old version (v0.11)

```
1  ...
2  @main.entry = ... c"main,entry\0A\00"
3  @main.if.then = ... c"main,if.then\0A\00"
4  @main.if.else = ... c"main,if.else\0A\00"
5  @main.if.end = ... c"main,if.end\0A\00"
6  ...
7  define i32 @main() {
8  entry:
9      call @printf(... @main.entry, ...) ;inst
10     ...
11  if.then:
12      call @printf(... @main.if.then, ...) ;inst
13     ...
14  if.else:
15      call @printf(... @main.if.else, ...) ;inst
16     ...
17  if.end:
18      call @printf(... @main.if.end, ...) ;inst
19     ...
20 }
21 ...
```

Figure: Basic block instrumentation. An instrumentation point is labeled with comment “;inst”.

Offline phase

get LLVM IR for $v_0.c$

instrument $v_0.ll$



test-coverage for $v_0.ll$



create forward map

Offline phase

get LLVM IR for $v_0.c$

instrument $v_0.ll$

test-coverage for $v_0.ll$



create forward map

Test-suite and test-coverage map

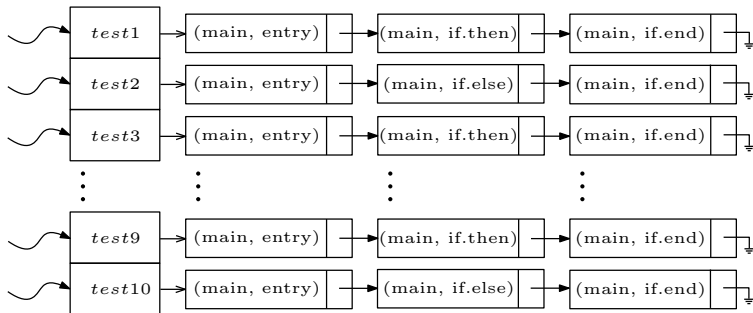


Figure: Example: test-case dependency graph (test-coverage map).

Test-suite and test-coverage map

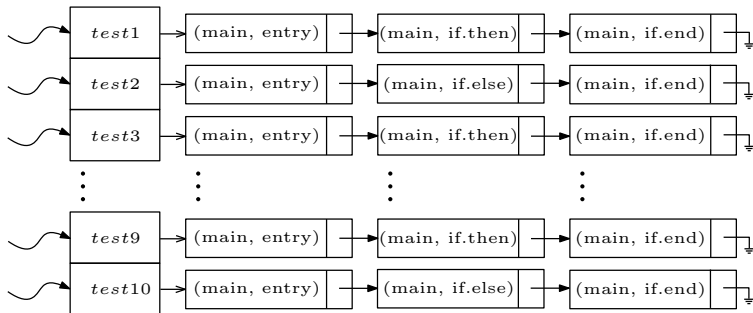


Figure: Example: test-case dependency graph (test-coverage map).

The test-suite consists of ten test-cases *test1* through *test10*, with odd and even numbered tests containing integer values $\{1, 3, 5, 7, 9\}$ and $\{-2, -4, -6, -8, -10\}$, respectively.

Classwork: find the test-coverage map for *prog_v0.c*

```
1 // prog_v0.c
2 #include<stdio.h>
3 int g=5;
4 ...
5 int main()
6 {
7     int n, b;
8     scanf("%d", &n);
9     if (n >= 0)
10    {
11        if(n % 2 == 0)
12            g=n/2;
13        else
14            g=n/3;
15    }
16    else
17    {
18        n=g*2;
19    }
20    return 0;
21 }
```

```
1 // prog_v1.c
2 #include<stdio.h>
3 int g=7; //change 1
4 ...
5 int main()
6 {
7     int n, b;
8     scanf("%d", &n);
9     if (n >= 0)
10    {
11        if(n % 2 == 0)
12            g=n/4; //change 2
13        else
14            g=n/3;
15    }
16    else
17    {
18        n=g*2;
19    }
20    return 0;
21 }
```

8 test-cases: {2, -67, 3, 20, 40, 15, 34, -56}.

Classwork: find the test-coverage map for *prog_v0.c*

In terms of source-code line³ coverage...

Forward map	t_1	→	l_3	l_7	l_8	l_9	l_{12}	l_{20}	(2)
	t_2	→	l_3	l_7	l_8	l_9	l_{18}	l_{20}	(-67)
	t_3	→	l_3	l_7	l_8	l_9	l_{14}	l_{20}	(3)
	t_4	→	l_3	l_7	l_8	l_9	l_{12}	l_{20}	(20)
	t_5	→	l_3	l_7	l_8	l_9	l_{12}	l_{20}	(40)
	t_6	→	l_3	l_7	l_8	l_9	l_{14}	l_{20}	(15)
	t_7	→	l_3	l_7	l_8	l_9	l_{12}	l_{20}	(34)
	t_8	→	l_3	l_7	l_8	l_9	l_{18}	l_{20}	(-56)

³Only the interesting ones are shown.